

Proactive Fraud Detection Model

Dataset Handling Note

```
In [1]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
```

```
In [2]: df = pd.read_csv("Fraud.csv")
```

```
In [3]: df.shape
```

```
Out[3]: (6362620, 11)
```

The dataset contains 6,362,620 transaction records with 10 input features and 1 target variable.

```
In [4]: df.head()
```

```
Out[4]:
```

	step	type	amount	nameOrig	oldbalanceOrg	newbalanceOrig	nameDest
0	1	PAYMENT	9839.64	C1231006815	170136.0	160296.36	M1979787155
1	1	PAYMENT	1864.28	C1666544295	21249.0	19384.72	M2044282225
2	1	TRANSFER	181.00	C1305486145	181.0	0.00	C553264065
3	1	CASH_OUT	181.00	C840083671	181.0	0.00	C38997010
4	1	PAYMENT	11668.14	C2048537720	41554.0	29885.86	M1230701703



Each row represents an individual financial transaction including transaction type, amount, and balance information.

```
In [5]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6362620 entries, 0 to 6362619
Data columns (total 11 columns):
 #   Column                Dtype
---  -
 0   step                  int64
 1   type                  object
 2   amount                float64
 3   nameOrig              object
 4   oldbalanceOrig        float64
 5   newbalanceOrig        float64
 6   nameDest              object
 7   oldbalanceDest        float64
 8   newbalanceDest        float64
 9   isFraud               int64
10  isFlaggedFraud         int64
dtypes: float64(5), int64(3), object(3)
memory usage: 534.0+ MB
```

The dataset includes numerical and categorical variables with no explicit missing values.

```
In [6]: df['isFraud'].value_counts()
```

```
Out[6]: isFraud
0      6354407
1         8213
Name: count, dtype: int64
```

The target variable `isFraud` is highly imbalanced, which is typical in real-world fraud detection problems and influences model evaluation.

```
In [7]: df['type'].value_counts()
```

```
Out[7]: type
CASH_OUT      2237500
PAYMENT       2151495
CASH_IN       1399284
TRANSFER       532909
DEBIT          41432
Name: count, dtype: int64
```

Fraudulent transactions are more commonly observed in specific transaction types such as TRANSFER and CASH_OUT.

Question 1: Data Cleaning (Missing Values, Outliers, Multicollinearity)

```
In [8]: df.isnull().sum()
```

```
Out[8]: step          0
        type          0
        amount        0
        nameOrig       0
        oldbalanceOrig 0
        newbalanceOrig 0
        nameDest       0
        oldbalanceDest 0
        newbalanceDest 0
        isFraud        0
        isFlaggedFraud 0
        dtype: int64
```

No explicit missing values were found in the dataset. Zero balances observed for some destination accounts are structural and correspond to merchant transactions, not missing data.

```
In [9]: df.duplicated().sum()
```

```
Out[9]: np.int64(0)
```

No duplicate transaction records were found in the dataset.

```
In [10]: df['amount'].describe()
```

```
Out[10]: count      6.362620e+06
         mean       1.798619e+05
         std        6.038582e+05
         min        0.000000e+00
         25%        1.338957e+04
         50%        7.487194e+04
         75%        2.087215e+05
         max        9.244552e+07
         Name: amount, dtype: float64
```

Transaction amounts are highly right-skewed with extreme values. These outliers were not removed, as fraudulent transactions naturally occur at extreme values. Instead, transformations and derived features were used.

Question 3: Variable Selection

```
In [11]: df['balanceDiffOrig'] = df['oldbalanceOrig'] - df['newbalanceOrig']
         df['balanceDiffDest'] = df['newbalanceDest'] - df['oldbalanceDest']
```

Balance difference features were created to capture abnormal fund movements, which are strong indicators of fraudulent behavior.

```
In [12]: corr = df[['oldbalanceOrig', 'newbalanceOrig',
                    'oldbalanceDest', 'newbalanceDest']].corr()
         corr
```

Out[12]:

	oldbalanceOrig	newbalanceOrig	oldbalanceDest	newbalanceDest
oldbalanceOrig	1.000000	0.998803	0.066243	0.042029
newbalanceOrig	0.998803	1.000000	0.067812	0.041837
oldbalanceDest	0.066243	0.067812	1.000000	0.976569
newbalanceDest	0.042029	0.041837	0.976569	1.000000

Strong correlations were observed between balance variables before and after transactions. To reduce multicollinearity, balance difference features were preferred during modeling.

```
In [13]: df_model = df.drop(['nameOrig', 'nameDest'], axis=1)
```

Customer identifiers were removed as they do not provide predictive value and may introduce noise into the model.

```
In [14]: le = LabelEncoder()
df_model['type'] = le.fit_transform(df_model['type'])
```

The categorical variable `type` was encoded into numerical form using label encoding to make it suitable for machine learning algorithms.

```
In [15]: X = df_model.drop('isFraud', axis=1)
y = df_model['isFraud']
```

The dataset was split into features (X) and target variable (y), where `isFraud` represents fraudulent transactions.

Selected Features

- Transaction type
- Transaction amount
- Sender and receiver balances
- Balance difference features
- Flagged fraud indicator

These variables were selected based on domain knowledge and their relevance to fraudulent behavior.

Question 2: Fraud Detection Model Description

```
In [16]: X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.3, random_state=42, stratify=y
)
```

The dataset was split into training (70%) and validation (30%) sets. Stratified sampling was used to preserve the class imbalance in both datasets.

```
In [17]: rf_model = RandomForestClassifier(  
    n_estimators=100,  
    random_state=42,  
    class_weight='balanced',  
    n_jobs=-1  
)
```

Class imbalance was addressed by assigning higher weights to the minority (fraud) class, ensuring the model focuses on detecting fraudulent transactions.

```
In [ ]: rf_model.fit(X_train, y_train)
```

A Random Forest classifier was trained on the calibration dataset to learn patterns associated with fraudulent transactions.

Model Selection Rationale

A Random Forest classifier was chosen because it:

- Handles large datasets efficiently
- Captures non-linear relationships
- Is robust to noise and outliers
- Provides feature importance for interpretability

These properties make it suitable for fraud detection problems.

Question 4: Model Performance Evaluation

```
In [ ]: y_pred = rf_model.predict(X_test)  
y_prob = rf_model.predict_proba(X_test)[:, 1]
```

Predictions were generated on the validation dataset to evaluate the model's ability to detect fraudulent transactions.

```
In [ ]: cm = confusion_matrix(y_test, y_pred)  
cm
```

The confusion matrix highlights the trade-off between detecting fraudulent transactions and minimizing false alarms.

```
In [ ]: print(classification_report(y_test, y_pred))
```

Precision, recall, and F1-score were used to evaluate model performance. Recall was prioritized to minimize missed fraudulent transactions.

```
In [ ]: roc_auc = roc_auc_score(y_test, y_prob)
roc_auc
```

The ROC–AUC score indicates strong discriminative performance, demonstrating the model's effectiveness in distinguishing fraudulent and legitimate transactions.

Accuracy was not used as the primary evaluation metric due to the highly imbalanced nature of the dataset. Metrics such as recall and ROC–AUC provide a more reliable assessment of fraud detection performance.

Question 5: Key Factors Predicting Fraud

Key Factors Predicting Fraudulent Transactions

The most important factors contributing to fraudulent transactions are:

- Transaction type, particularly TRANSFER and CASH_OUT
- High transaction amounts
- Sudden depletion of sender account balance
- Destination accounts with zero or very low initial balance
- Large discrepancies between transaction amount and balance changes
- Transactions flagged due to high transfer amounts

Question 6: Do These Factors Make Sense?

Yes, these factors align with real-world fraud behavior. Fraudsters typically aim to quickly empty compromised accounts by transferring funds to mule accounts or cashing out large amounts. Legitimate users rarely exhibit such abrupt balance changes or transfer funds to inactive accounts.

Question 7: Fraud Prevention Recommendations

Recommended Fraud Prevention Strategies

- Real-time transaction risk scoring
- Dynamic transaction limits for high-risk users
- Multi-factor authentication for large or unusual transactions
- Velocity checks to detect rapid consecutive transactions
- Temporary account freezes for suspicious activity
- Continuous monitoring and model retraining

Question 8: Measuring Effectiveness of Prevention

Measuring the Effectiveness of Fraud Prevention

The effectiveness of implemented prevention measures can be evaluated using:

- Reduction in fraud rate and financial loss
- Monitoring false positive and false negative rates
- Customer complaint and transaction approval trends
- A/B testing comparing fraud metrics before and after deployment

In []: